James Finn SUNY Korea
CSE 114 Spring 26

Assignment 3—Recursion

Due: 27 April

Submit your work as one .zip file on Brightspace.

Each .java file must have your name (and the name of your partner if you work with one) in a comment at the top.

In this assignment, you will practice writing recursive functions. Remember: when you use recursion, you should be sure that

a) When a function calls itself recursively, it is calling itself on a *smaller problem*.

b) When the problem is small enough, the function must calculate the result directly, without making any more recursive calls.

Problem 1

Write a recursive Java function that returns the reverse of a string. This is the interface:

```
/*
     function: reverseString
     Returns the reverse of its argument s.
     Example: reverse("pigs") returns "sgip".
 */
public static String reverseString(String s);
```

To do this recursively, you can think of the string as consisting of its first character and the substring that is the rest of the string without the first character. You can reverse the rest of the string recursively then append the first character to the end. Example: for "pigs", you could reverse "igs" recursively to get "sgi" then append the 'p' to the end to get "sgip".

Include a main function that tests the **`reverseString`** function.

Problem 2

In this problem, you will write a recursive version of the reverse array function from Problem 6 of the midterm:

```
/*
     function; reverseArray
     Reverses the contents of a[low] up to a[high].
     Example: if a is {2, 4, 6, 8},
     then after calling reverse(a, 0, 3),
     a would be {8, 6, 4, 2}.
 */
public static void reverseArray(
     int [] a, int low, int high);
```

To do this recursively, you can't simply break the array into two parts, the first item and the rest of the array (why not?) You need to work using the second method suggested in the midterm problem: work from the ends inward. You can swap the endpoints **`a[low]`** and **`a[high]`**, then reverse the inner part of the array recursively.

Include a main program to test your function.

Problem 3

For this problem you will write a recursive function to calculate exponents that uses a much faster algorithm than the one we used in the recursion handout. In that implementation, **`expon(base, exp)`** calls **`expon(base, exp-1)`**. This means that the number of recursive calls is proportional to the value of **`exp`**: if **`exp`** doubles, the runtime doubles. In this new algorithm the number of recursive calls is only the log of **`exp`**. This is like the difference between linear search and binary search.

The algorithm is based on the fact that $(n^a)^b = n^{ab}$. If *exp* is even, say 2*x*, then $n^{2x} = (n^x)^2$. We can calculate n^x recursively, then multiply it by itself to get the result. If *exp* is odd, say 2*x*+1, then $n^{2x+1} = (n^x)^2(n)$. Again, n^x can be calculated recursively and used to complete the result. In the recursive calls the exponent is at most half the size of the argument *exp*, so we get log *n* behavior.

```
public static long expon(long base, long exp);
```

Include a main program to test your function.